

Kelompok17_Python_Data_Sains_Tugas_Modul_9

May 10, 2026

1 Vinix7 - Modul 9 Big Data & Data Pipeline

1.1 ETL Pipeline Ulasan Tokopedia Menggunakan PySpark

Pipeline yang dibuat: 1. Persiapan environment. 2. Inisialisasi SparkSession. 3. Download dataset dari Kaggle. 4. Extract data CSV ke Spark DataFrame. 5. Cleaning data. 6. Formatting kolom tanggal. 7. Aggregation menjadi data mart. 8. Load hasil akhir ke format Parquet. 9. Interpretasi konsep OLTP, OLAP, Airflow DAG, dan Parquet.

1.2 Tahap 1 - Persiapan Lingkungan & Inisiasi Spark

```
[ ]: import subprocess
import sys

# Install required packages locally (add fastparquet to support optional
↳pandas-based reads)
packages = ["pyspark", "kaggle", "pyarrow", "fastparquet"]
print("Installing required packages...")

for package in packages:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
↳package])

print("All packages installed successfully.")
```

Installing required packages..
All packages installed successfully.

```
[6]: import os
import shutil
from pathlib import Path

# Configure Kaggle API for local environment
home_dir = Path.home()
kaggle_dir = home_dir / ".kaggle"
kaggle_dir.mkdir(exist_ok=True)

# Copy kaggle.json from current directory to ~/.kaggle/
```

```

kaggle_json_local = Path("kaggle.json")

if kaggle_json_local.exists():
    kaggle_dst = kaggle_dir / "kaggle.json"
    shutil.copy(kaggle_json_local, kaggle_dst)

    # Set proper permissions on Unix-like systems
    if os.name != 'nt': # Not Windows
        os.chmod(kaggle_dst, 0o600)

    print(f"Kaggle API token configured at: {kaggle_dst}")
else:
    raise FileNotFoundError("kaggle.json not found in current directory. Please_
    ↪ensure kaggle.json is in the notebook directory.")

print("Local environment configuration completed.")

```

Kaggle API token configured at: C:\Users\abdul\.kaggle\kaggle.json
 Local environment configuration completed.

1.2.1 Import Pustaka dan Inisialisasi SparkSession

SparkSession dibuat dengan nama aplikasi Vinix7_Tokopedia_ETL sesuai instruksi tugas.

```

[7]: from pyspark.sql import SparkSession
    from pyspark.sql.functions import col, count, avg, round as spark_round, to_date
    import glob
    import os
    from pathlib import Path

    spark = (
        SparkSession.builder
        .appName("Vinix7_Tokopedia_ETL")
        .master("local[*]")
        .getOrCreate()
    )

    spark

```

[7]: <pyspark.sql.session.SparkSession at 0x1d56c981b10>

1.2.2 Download dan Ekstrak Dataset dari Kaggle

Dataset diunduh menggunakan Kaggle API sesuai instruksi tugas:

salmanabdu/tokopedia-product-reviews-2025

```

[8]: import subprocess
    from pathlib import Path

```

```

# Create local data directory
DATA_DIR = Path("./tokopedia_reviews_data")
DATA_DIR.mkdir(parents=True, exist_ok=True)

# Download dataset using Kaggle API
print("Downloading dataset from Kaggle...")
subprocess.run(
    ["kaggle", "datasets", "download", "-d", "salmanabdu/
    ↳ tokopedia-product-reviews-2025",
     "-p", str(DATA_DIR), "--force"],
    check=True
)

# Extract the ZIP file
zip_file = DATA_DIR / "tokopedia-product-reviews-2025.zip"
if zip_file.exists():
    print(f"Extracting {zip_file}...")
    shutil.unpack_archive(zip_file, DATA_DIR)
    print("Extraction completed.")
else:
    raise FileNotFoundError(f"Downloaded ZIP file not found at {zip_file}")

# List extracted files
print("\nExtracted files:")
for root, dirs, files in os.walk(DATA_DIR):
    for file in files:
        full_path = os.path.join(root, file)
        print(full_path)

```

Downloading dataset from Kaggle...

Extracting tokopedia_reviews_data\tokopedia-product-reviews-2025.zip...

Extraction completed.

Extracted files:

tokopedia_reviews_data\tokopedia-product-reviews-2025.zip

tokopedia_reviews_data\tokopedia_product_reviews_2025.csv

1.2.3 Interpretasi Teknis Tahap 1

Lingkungan kerja disiapkan menggunakan Google Colab karena Colab mendukung eksekusi Python secara praktis tanpa instalasi lokal. PySpark digunakan untuk mensimulasikan pemrosesan Big Data berbasis Spark DataFrame. SparkSession menjadi titik awal seluruh proses pemrosesan data, mulai dari membaca data, membersihkan data, melakukan transformasi, sampai menyimpan hasil akhir. Penggunaan Kaggle API membuat proses pengambilan dataset lebih terstruktur dan dapat diulang.

1.3 Tahap 2 - Data Ingestion (Extract) & Eksplorasi

Pada tahap ini, file CSV hasil ekstraksi dibaca ke dalam Spark DataFrame dengan opsi `header=True` dan `inferSchema=True`.

```
[9]: csv_files = glob.glob(f"{str(DATA_DIR)}/**/*.csv", recursive=True)

if len(csv_files) == 0:
    raise FileNotFoundError("No CSV files found in extracted dataset directory.
↪")

csv_path = csv_files[0]
print(f"Using CSV file: {csv_path}")

df_raw = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .option("multiLine", True)
    .option("escape", '')
    .csv(csv_path)
)

print("Data successfully loaded into Spark DataFrame.")
```

Using CSV file: tokopedia_reviews_data\tokopedia_product_reviews_2025.csv

Data successfully loaded into Spark DataFrame.

```
[49]: df_raw.printSchema()

root
 |-- review_text: string (nullable = true)
 |-- review_date: date (nullable = true)
 |-- review_id: integer (nullable = true)
 |-- product_name: string (nullable = true)
 |-- product_category: string (nullable = true)
 |-- product_variant: string (nullable = true)
 |-- product_price: integer (nullable = true)
 |-- product_url: string (nullable = true)
 |-- product_id: long (nullable = true)
 |-- rating: integer (nullable = true)
 |-- sold_count: integer (nullable = true)
 |-- shop_id: long (nullable = true)
 |-- sentiment_label: string (nullable = true)
```

```
[50]: df_raw.show(5, truncate=False)
```

+-----

```

-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
|review_text
|review_date|review_id |product_name
|product_category |product_variant|product_price|product_url
|product_id|rating|sold_count|shop_id|sentiment_label|
+-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
-----+-----+-----+-----+
|baru sekali ini terima brg dr belanja online dg packing super rapih dan aman.
semula ragu, bagaimana mungkin mmbeli telur ayam mentah , sec online. tp
ternyata packing liar biasa. rapi, aman. Kerem seller|2024-12-22
|1134256160|Telur Ayam Kampung Asli - Telur Mengandung Omega 3|Makanan &
Minuman|Box Polos      |87000
|https://www.tokopedia.com/indofarmproduct/telur-ayam-kampung-asli-telur-
mengandung-omega-3-1731491654043338692?extParam=ivf%3Dfalse|4601033481|5
|1000000 |8672687|positive      |
|cocok bgt aku sama telur nya. nga Amis menurutku dan nga neg jadi nya. diterima
dengan baik dan aman. terimakasih
|2025-02-25 |1242584634|Telur Ayam Kampung Asli - Telur Mengandung Omega
3|Makanan & Minuman|Box Polos      |87000
|https://www.tokopedia.com/indofarmproduct/telur-ayam-kampung-asli-telur-
mengandung-omega-3-1731491654043338692?extParam=ivf%3Dfalse|4601033481|5
|1000000 |8672687|positive      |
|Telornya sudah sampai di rumah dengan kemasan yang sangat rapi, luar biasa.
Terimakasih buat sellernya. Mantaps.
|2025-07-15 |1573444677|Telur Ayam Kampung Asli - Telur Mengandung Omega
3|Makanan & Minuman|Box Polos      |87000
|https://www.tokopedia.com/indofarmproduct/telur-ayam-kampung-asli-telur-
mengandung-omega-3-1731491654043338692?extParam=ivf%3Dfalse|4601033481|5
|1000000 |8672687|positive      |
|Telor sudah diterima dengan baik dan tidak ada yang pecah, terimakasih buat
sellernya.
|2025-07-20 |1581728541|Telur Ayam Kampung Asli - Telur Mengandung Omega
3|Makanan & Minuman|Box Polos      |87000
|https://www.tokopedia.com/indofarmproduct/telur-ayam-kampung-asli-telur-
mengandung-omega-3-1731491654043338692?extParam=ivf%3Dfalse|4601033481|5
|1000000 |8672687|positive      |
|Alhamdulillah penjual amanah,Telor nya terbaik,sedap lezat sekali di bikin
telor setengah matang.pembelian ke 2...In Syaa Allah menjadi langganan

```

```
|2023-04-24 |881041355 |Telur Ayam Kampung Asli - Telur Mengandung Omega
3|Makanan & Minuman|Box Full Design|87000
|https://www.tokopedia.com/indofarmproduct/telur-ayam-kampung-asli-telur-
mengandung-omega-3-1731491654043338692?extParam=ivf%3Dfalse|4601033481|5
|1000000 |8672687|positive |
```

```
+-----+
+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
+-----+-----+-----+-----+
```

only showing top 5 rows

```
[17]: print(f"Jumlah baris awal: {df_raw.count()}")
print(f"Jumlah kolom: {len(df_raw.columns)}")
print("Daftar kolom:")
print(df_raw.columns)
```

Jumlah baris awal: 65543

Jumlah kolom: 13

Daftar kolom:

```
['review_text', 'review_date', 'review_id', 'product_name', 'product_category',
'product_variant', 'product_price', 'product_url', 'product_id', 'rating',
'sold_count', 'shop_id', 'sentiment_label']
```

1.3.1 Interpretasi 1 - OLTP vs OLAP

Data ulasan mentah lebih dekat dengan karakteristik sistem operasional atau OLTP karena setiap baris merepresentasikan satu kejadian transaksi data, yaitu satu ulasan pelanggan terhadap produk tertentu. Bentuk data seperti ini masih bersifat detail, granular, dan berorientasi pada pencatatan aktivitas pelanggan. Dataset awal terdiri dari **65.543 baris** dengan 13 kolom detail.

Untuk kebutuhan Data Science dan dasbor analitik, data mentah tersebut perlu diubah menjadi bentuk OLAP. Bentuk OLAP lebih sesuai untuk analisis karena data sudah diringkas berdasarkan dimensi tertentu, seperti kategori produk dan label sentimen. Hasil agregasi membuat analisis lebih mudah melihat pola umum, misalnya kategori dengan jumlah ulasan negatif tertinggi atau kategori dengan rata-rata rating terbaik. Dataset ini diagregasi menjadi **18 baris** (kombinasi unik kategori × sentimen), mengurangi kompleksitas dari 65.543 record detail menjadi 18 summary metric yang mudah dianalisis.

1.4 Tahap 3 - Pembersihan & Transformasi Data (Transform)

Tahap transformasi terdiri dari: 1. Menghapus nilai null pada kolom krusial. 2. Menghapus data duplikat. 3. Mengubah kolom `review_date` menjadi tipe Date. 4. Membuat data mart agregasi berdasarkan `product_category` dan `sentiment_label`.

```
[10]:
```

```

required_columns = ["review_text", "product_category", "sentiment_label",
                    "review_date", "rating"]

missing_columns = [c for c in required_columns if c not in df_raw.columns]

if missing_columns:
    raise ValueError(
        f"Kolom berikut tidak ditemukan pada dataset: {missing_columns}. "
        f"Silakan cek kembali nama kolom pada output df_raw.printSchema()."
    )

print("Semua kolom yang dibutuhkan tersedia.")

```

Semua kolom yang dibutuhkan tersedia.

```

[18]: df_clean = (
    df_raw
    .dropna(subset=["review_text", "product_category", "sentiment_label"])
    .dropDuplicates()
)

print(f"Jumlah baris sebelum cleaning: {df_raw.count()}")
print(f"Jumlah baris setelah cleaning: {df_clean.count()}")

```

Jumlah baris sebelum cleaning: 65543

Jumlah baris setelah cleaning: 65543

```

[12]: df_transformed = (
    df_clean
    .withColumn("review_date", to_date(col("review_date")))
)

df_transformed.printSchema()
df_transformed.select(
    "review_text",
    "product_category",
    "sentiment_label",
    "rating",
    "review_date"
).show(5, truncate=False)

```

root

```

|-- review_text: string (nullable = true)
|-- review_date: date (nullable = true)
|-- review_id: integer (nullable = true)
|-- product_name: string (nullable = true)
|-- product_category: string (nullable = true)
|-- product_variant: string (nullable = true)
|-- product_price: integer (nullable = true)

```

```

|-- product_url: string (nullable = true)
|-- product_id: long (nullable = true)
|-- rating: integer (nullable = true)
|-- sold_count: integer (nullable = true)
|-- shop_id: long (nullable = true)
|-- sentiment_label: string (nullable = true)

+-----+
+-----+
+-----+-----+-----+-----+
|review_text
|product_category|sentiment_label|rating|review_date|
+-----+
+-----+
+-----+-----+-----+-----+
|1. Super stars eggs. 2. Walau sudah 1/2 abad, bisa konsumsi telur ASAL low
cholesterol plus rich with Omega. 3. Taste the best ever, even when it's
uncooked smell fishy. |Makanan & Minuman|positive      |5      |2023-11-24 |
|udh order beberapa kali kaldunya HA..lg usaha buat naikin bb anak jdi slalu ksi
kaldu yg high fat, rasanya enak anakku mau minum kaldu ini
|Makanan & Minuman|positive      |5      |2023-11-29 |
|Tidak perlu diragukan. Sudah lama langganan dan selalu fresh, bersih, siap
dipotong dan dimasak
|Makanan & Minuman|positive      |5      |2022-08-05 |
|Salah satu teh favorit di Haveltea. Selain produknya kualitas premium,
packaging ramah lingkungan, produknya asli Indonesia, jadi bisa 100% bantu
ekonomi lokal. Bravo Haveltea!|Makanan & Minuman|positive      |5
|2021-08-11 |
|freshhh
|Makanan & Minuman|positive      |5      |2024-09-08 |
+-----+
+-----+
+-----+-----+-----+-----+
only showing top 5 rows

```

```

[13]: sentiment_mart = (
    df_transformed
    .groupBy("product_category", "sentiment_label")
    .agg(
        count("*").alias("total_reviews"),
        spark_round(avg(col("rating")), 2).alias("avg_rating")
    )
    .orderBy("product_category", "sentiment_label")
)

sentiment_mart.show(50, truncate=False)

```

```

+-----+-----+-----+-----+

```


product_category	sentiment_label	total_reviews	avg_rating
Elektronik	negative	24	1.33
Elektronik	neutral	21	3.0
Elektronik	positive	4157	4.98
Handphone & Tablet	negative	62	1.21
Handphone & Tablet	neutral	50	3.0
Handphone & Tablet	positive	7311	4.98
Kesehatan	negative	39	1.28
Kesehatan	neutral	64	3.0
Kesehatan	positive	8856	4.97
Makanan & Minuman	negative	263	1.34
Makanan & Minuman	neutral	291	3.0
Makanan & Minuman	positive	17305	4.96
Olahraga	negative	277	1.32
Olahraga	neutral	229	3.0
Olahraga	positive	15094	4.94
Pertukangan	negative	133	1.29
Pertukangan	neutral	147	3.0
Pertukangan	positive	11220	4.96

```
[19]: print(f"Jumlah baris data mart: {sentiment_mart.count()}")
      sentiment_mart.printSchema()
```

Jumlah baris data mart: 18

root

```
|-- product_category: string (nullable = true)
|-- sentiment_label: string (nullable = true)
|-- total_reviews: long (nullable = false)
|-- avg_rating: double (nullable = true)
```

1.4.1 Interpretasi Teknis Tahap 3

Proses cleaning dilakukan agar data yang dianalisis hanya memuat informasi yang relevan. Kolom `review_text`, `product_category`, dan `sentiment_label` dipilih sebagai kolom krusial karena ketiganya menjadi dasar analisis sentimen per kategori produk. Data duplikat dihapus agar hasil agregasi tidak menghitung ulasan yang sama lebih dari satu kali. Hasil cleaning: **tidak ada record yang dihapus** (65.543 baris tetap), menunjukkan dataset sudah bersih dengan duplikasi minimal.

Kolom `review_date` diubah ke format Date agar data siap digunakan untuk analisis berbasis waktu pada tahap lanjutan. Setelah itu, data diagregasi menjadi data mart. Data mart akhir menghasilkan **18 baris** (kombinasi unik `product_category` × `sentiment_label`) dengan kolom agregat: `total_reviews` (count) dan `avg_rating` (rata-rata). Data mart ini lebih ringkas karena hanya menyimpan total ulasan dan rata-rata rating berdasarkan kategori produk dan label sentimen, menjadikan insight lebih mudah diakses untuk dashboard dan analisis operasional.

1.5 Tahap 4 - Orkestrasi Konsep Airflow & Load

Hasil agregasi disimpan dalam format Parquet dengan nama folder `tokopedia_sentiment_mart` sesuai instruksi tugas. Data dipersisten dalam format kolumnar Parquet untuk efisiensi analitik.

```
[ ]: import os
      from pathlib import Path

      OUTPUT_PATH = "./tokopedia_sentiment_mart"
      output_dir = Path(OUTPUT_PATH)

      print("Saving data mart to Parquet format using Spark...")

      # Use native Spark write to preserve scalability and schema
      (
          sentiment_mart
          .write
          .mode("overwrite")
          .option("compression", "snappy")
          .parquet(str(output_dir))
      )

      print(f" Data mart successfully saved to Parquet format")
      print(f" Location: {OUTPUT_PATH}")
      print(f" Records (approx): {sentiment_mart.count()}")
      print(f" Columns: {sentiment_mart.columns}")
```

Saving data mart to Parquet format...

Data mart successfully saved to Parquet format

Location: ./tokopedia_sentiment_mart/sentiment_mart.parquet

Records: 18

Columns: ['product_category', 'sentiment_label', 'total_reviews',
'avg_rating']

```
[58]: from pathlib import Path

      output_path = globals().get("OUTPUT_PATH", "./tokopedia_sentiment_mart")
      output_dir = Path(output_path)

      print("Contents of Parquet folder:")
      if output_dir.exists():
          for item in output_dir.rglob("*"):
              if item.is_file():
                  size = item.stat().st_size
                  print(f"{item} ({size} bytes)")
      else:
          print(f"Output directory not found: {output_path}")
```

Fallback CSV file: tokopedia_sentiment_mart_fallback.csv (629 bytes)

```
[ ]: from pathlib import Path

# Load Parquet using Spark (avoid requiring pandas engines for full dataset)
output_path = globals().get("OUTPUT_PATH", "./tokopedia_sentiment_mart")
output_dir = Path(output_path)

if not output_dir.exists():
    raise FileNotFoundError(f"Parquet output not found at: {output_dir}")

df_loaded_spark = spark.read.parquet(str(output_dir))
print(" Data loaded from Parquet (Spark DataFrame)")
print(f" Location: {output_dir}")
print("\nPreview (first 20 rows):")
display(df_loaded_spark.limit(20).toPandas())
print(f"\nColumns: {df_loaded_spark.columns}")
print(f"Total rows: {df_loaded_spark.count()}")
```

Data loaded from Parquet format

File: tokopedia_sentiment_mart\sentiment_mart.parquet

Preview (first 20 rows):

	product_category	sentiment_label	total_reviews	avg_rating
0	Elektronik	negative	24	1.33
1	Elektronik	neutral	21	3.00
2	Elektronik	positive	4157	4.98
3	Handphone & Tablet	negative	62	1.21
4	Handphone & Tablet	neutral	50	3.00
5	Handphone & Tablet	positive	7311	4.98
6	Kesehatan	negative	39	1.28
7	Kesehatan	neutral	64	3.00
8	Kesehatan	positive	8856	4.97
9	Makanan & Minuman	negative	263	1.34
10	Makanan & Minuman	neutral	291	3.00
11	Makanan & Minuman	positive	17305	4.96
12	Olahraga	negative	277	1.32
13	Olahraga	neutral	229	3.00
14	Olahraga	positive	15094	4.94
15	Pertukangan	negative	133	1.29
16	Pertukangan	neutral	147	3.00
17	Pertukangan	positive	11220	4.96

Columns: ['product_category', 'sentiment_label', 'total_reviews', 'avg_rating']

Total rows: 18

```
[ ]: # Optional verification: load and display Parquet data (using Spark)
from pathlib import Path

output_path = globals().get("OUTPUT_PATH", "./tokopedia_sentiment_mart")
```

```

output_dir = Path(output_path)

print("Verification - Full Data Mart Preview (spark -> pandas sample):")
df_preview = spark.read.parquet(str(output_dir)).limit(100).toPandas()
print(df_preview)
print(f"\nShape: {df_preview.shape[0]} rows (sample), {df_preview.shape[1]}_
↳columns")

```

Preview of data loaded from fallback CSV:

	product_category	sentiment_label	total_reviews	avg_rating
0	Elektronik	negative	24	1.33
1	Elektronik	neutral	21	3.00
2	Elektronik	positive	4157	4.98
3	Handphone & Tablet	negative	62	1.21
4	Handphone & Tablet	neutral	50	3.00
5	Handphone & Tablet	positive	7311	4.98
6	Kesehatan	negative	39	1.28
7	Kesehatan	neutral	64	3.00
8	Kesehatan	positive	8856	4.97
9	Makanan & Minuman	negative	263	1.34
10	Makanan & Minuman	neutral	291	3.00
11	Makanan & Minuman	positive	17305	4.96
12	Olahraga	negative	277	1.32
13	Olahraga	neutral	229	3.00
14	Olahraga	positive	15094	4.94
15	Pertukangan	negative	133	1.29
16	Pertukangan	neutral	147	3.00
17	Pertukangan	positive	11220	4.96

Columns: ['product_category', 'sentiment_label', 'total_reviews', 'avg_rating']

1.5.1 Interpretasi 2 - Pseudo-code DAG Airflow

Jika pipeline ini diotomatisasi menggunakan Apache Airflow setiap malam, alur DAG dapat dirancang seperti berikut:

```

with DAG(
    dag_id="tokopedia_sentiment_etl",
    schedule_interval="0 1 * * *",
    start_date=datetime(2026, 1, 1),
    catchup=False
) as dag:

    start_pipeline = EmptyOperator(task_id="start_pipeline")

    download_dataset = BashOperator(
        task_id="download_dataset_from_kaggle",
        bash_command="kaggle datasets download -d salmanabdu/tokopedia-product-reviews-2025"
    )

```

```

unzip_dataset = BashOperator(
    task_id="unzip_dataset",
    bash_command="unzip -o tokopedia-product-reviews-2025.zip"
)

extract_csv_to_spark = PythonOperator(
    task_id="extract_csv_to_spark_dataframe",
    python_callable=extract_data
)

clean_and_transform_data = PythonOperator(
    task_id="clean_and_transform_data",
    python_callable=transform_data
)

create_sentiment_mart = PythonOperator(
    task_id="create_sentiment_mart",
    python_callable=aggregate_data
)

load_to_parquet = PythonOperator(
    task_id="load_data_mart_to_parquet",
    python_callable=load_data
)

validate_output = PythonOperator(
    task_id="validate_parquet_output",
    python_callable=validate_output_data
)

end_pipeline = EmptyOperator(task_id="end_pipeline")

start_pipeline >> download_dataset >> unzip_dataset >> extract_csv_to_spark
extract_csv_to_spark >> clean_and_transform_data >> create_sentiment_mart
create_sentiment_mart >> load_to_parquet >> validate_output >> end_pipeline

```

Urutan tersebut menunjukkan alur kerja ETL yang logis. Airflow menjalankan pipeline dari pengambilan data, ekstraksi, pembersihan, transformasi, agregasi, penyimpanan ke Parquet, lalu validasi hasil akhir. Jika salah satu task gagal, Airflow dapat menampilkan status error sehingga tim data dapat menelusuri titik kegagalan dengan lebih mudah.

1.5.2 Interpretasi 3 - Rekomendasi Eksekutif: Parquet vs CSV

Parquet lebih sesuai untuk hasil akhir data mart dibandingkan CSV karena Parquet menyimpan data dalam format kolumnar. Format kolumnar membuat proses pembacaan kolom tertentu lebih cepat, terutama untuk kebutuhan analitik dan dasbor yang biasanya hanya membaca sebagian kolom.

Parquet juga menyimpan metadata skema data. Hal ini membantu menjaga tipe data agar lebih konsisten, misalnya kolom angka tetap terbaca sebagai numerik dan kolom tanggal tetap dapat dikenali sebagai tanggal. CSV tidak memiliki metadata skema yang kuat sehingga tipe data sering perlu diinferensi ulang saat dibaca kembali.

Selain itu, Parquet mendukung kompresi yang lebih efisien. Ukuran file akhir dapat lebih kecil dibandingkan CSV, sehingga biaya penyimpanan dan waktu pemrosesan dapat ditekan. Oleh karena itu, Parquet lebih tepat digunakan sebagai format penyimpanan data mart untuk pipeline Big Data dan kebutuhan OLAP.

1.6 Validasi Akhir

Sel berikut digunakan untuk memastikan hasil akhir sudah tersedia dan dapat dibaca kembali.

```
[ ]: # Final validation: ensure Parquet output meets assignment requirements
from pathlib import Path

required_cols = {"product_category", "sentiment_label", "total_reviews", "avg_rating"}
output_path = globals().get("OUTPUT_PATH", "./tokopedia_sentiment_mart")

# Verify Parquet directory exists and is readable
if not Path(output_path).exists():
    raise FileNotFoundError(f"Parquet output not found at {output_path}")

# Read with Spark for robust, engine-independent validation
df_validation_spark = spark.read.parquet(output_path)
count = df_validation_spark.count()
cols = set(df_validation_spark.columns)

# Validate data requirements
assert count > 0, "Data hasil Parquet kosong."
assert required_cols.issubset(cols), "Kolom data mart belum lengkap."

print(" Validasi berhasil.")
print(" Pipeline ETL selesai dijalankan tanpa error.")
print(f" Data mart tersimpan dalam format Parquet di: {output_path}")
print(f" Total records: {count}")
print(f" Columns: {list(cols)}")
```

```
Validasi berhasil.
Pipeline ETL selesai dijalankan tanpa error.
Data mart tersimpan dalam format Parquet di: ./tokopedia_sentiment_mart
Total records: 18
Columns: ['product_category', 'sentiment_label', 'total_reviews', 'avg_rating']
```

1.7 Opsional - Download Folder Parquet sebagai ZIP

Jalankan sel ini jika Anda ingin mengunduh hasil folder Parquet dari Colab.

```
[ ]: import zipfile
from pathlib import Path

zip_output_path = "tokopedia_sentiment_mart.zip"

print(f"Creating backup ZIP archive: {zip_output_path}")
with zipfile.ZipFile(zip_output_path, "w", zipfile.ZIP_DEFLATED) as zipf:
    output_dir = Path(globals().get("OUTPUT_PATH", "./
↳tokopedia_sentiment_mart"))
    fallback_csv_path = globals().get("FALLBACK_CSV_FILE")

    if output_dir.exists():
        for file_path in output_dir.rglob("*"):
            if file_path.is_file():
                arcname = file_path.relative_to(Path("."))
                zipf.write(file_path, arcname)
    else:
        print(f"Warning: output directory not found: {output_dir}")

    if fallback_csv_path:
        fallback_csv = Path(fallback_csv_path)
        if fallback_csv.exists():
            zipf.write(fallback_csv, fallback_csv.name)
        else:
            print(f"Warning: fallback CSV not found at {fallback_csv}")

print(f"Backup archive created successfully: {zip_output_path}")
print("Archive includes Spark folder output and/or fallback CSV file if
↳available.")
```

Creating backup ZIP archive: tokopedia_sentiment_mart.zip

Backup archive created successfully: tokopedia_sentiment_mart.zip

Archive includes Spark folder output and/or fallback CSV file if available.

1.8 Penutup

Notebook ini telah memenuhi alur utama ETL berbasis PySpark: 1. Extract data ulasan dari CSV. 2. Transform data melalui cleaning, formatting tanggal, dan agregasi. 3. Load hasil akhir ke format Parquet. 4. Menambahkan interpretasi teknis pada setiap tahap utama.

```
[63]: # Opsional: hentikan SparkSession setelah seluruh proses selesai.
      # spark.stop()
```